

TELEMEDICINE ACCESSSS FOR RURAL HEALTHCARE IN NABHA

A Report

*Submitted in partial fulfilment of the
Requirements for the completion of*

THEME BASED PROJECT

BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

By

THAVIDABOINA PRAGATHI 1602-23-737-159
VANAMALA SHIVANI 1602-23-737-172
MANTHENA VYSHNAVI 1602-23-737-318

**Under the guidance of
Dr. Burgula Kezia Rani
Associate Professor**



Department of Information Technology

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE.

(Affiliated to Osmania University and Approved by AICTE)

Ibrahim Bagh, Hyderabad-31

2026

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE

(Affiliated to Osmania University and Approved by AICTE)

Ibrahim Bagh, Hyderabad-31

**Department of Information
Technology**



DECLARATION BY CANDIDATES

We, **T. Pragathi, V. Shivani, M. Vyshnavi** bearing hall ticket number, **1602-23-737-159, 1602-23-737-172, 1602-23-737-318**, hereby declare that the project report entitled **"TELEMEDICINE ACCESS FOR RURAL HEALTHCARE IN NABHA"** under the guidance of Dr. Burgula Kezia Rani, Associate Professor, Department of Information Technology, Vasavi College of Engineering, Hyderabad, is submitted in partial fulfillment of the requirement for the completion of Theme-based project, VI semester, Bachelor of Engineering in Information Technology.

This is a record of bonafide work carried out by us and the results embodied in this project report have not been submitted to any other institutes.

T. Pragathi, V. Shivani, M. Vyshnavi
1602-23-737-159, 1602-23-737-172, 1602-23-737-318

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE

(Affiliated to Osmania University and Approved by AICTE)

Ibrahim Bagh, Hyderabad-31

**Department of Information
Technology**



BONAFIDE CERTIFICATE

This is to certify that the project entitled “**TELEMEDICINE ACCESS FOR RURAL HEALTHCARE IN NABHA**” being submitted by **T. Pragathi, V. Shivani, M. Vyshanvi** bearing **1602-23-737-159, 1602-23-737-172, 1602-23-737-318**, in partial fulfillment of the requirements for the completion of Theme-based project of Bachelor of Engineering in Information Technology is a record of bonafide work carried out by them under my guidance.

Dr. Burgula Kezia Rani
Associate Professor

External Examiner

Dr. K. Ram Mohan Rao
Professor, HOD IT

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the project would not have been possible without the kind support and help of many individuals. We would like to extend our sincere thanks to all of them.

It is with immense pleasure that we would like to take the opportunity to express our humble gratitude to **Dr. Burgula Kezia Rani, Associate Professor, Information Technology** under whom we executed this project. Her constant guidance and willingness to share their vast knowledge made us understand this project and its manifestations in great depths and helped us to complete the assigned tasks.

We are very much thankful to **Dr. K. Ram Mohan Rao, Professor and HOD, Information Technology**, for his kind support and for providing necessary facilities to carry out the work.

We wish to convey our special thanks to **Dr. S. V. Ramana, Principal of Vasavi College of Engineering** for giving the required information in doing my project work. Not to forget, we thank all other faculty and non-teaching staff, and my friends who had directly or indirectly helped and supported me in completing my project in time.

We also express our sincere thanks to the Management for providing excellent facilities. Finally, we wish to convey our gratitude to our family who fostered all the facilities that we need.

ABSTRACT

The proposed system is a telemedicine platform designed to improve healthcare accessibility, particularly in rural areas where medical facilities and specialist availability are limited. The application enables users to consult doctors digitally, book appointments, analyze symptoms, and maintain medical records in an efficient and organized manner.

The system is developed using Flutter for the frontend to ensure cross-platform compatibility, Node.js with Express for the backend to handle application logic and APIs, and MongoDB for flexible and scalable data storage. It supports both online and offline appointment booking, allowing users to choose consultations based on their accessibility and convenience. A rule-based symptom checker is incorporated to assist users in identifying potential health conditions and recommending appropriate medical specialists. Additionally, the pharmacy module helps users locate nearby medicine availability, improving post-consultation accessibility.

To ensure data security and privacy, the system implements bcrypt for password hashing, JWT for authentication, and SHA-256 hashing for protecting sensitive medical data. The platform is designed with scalability in mind and includes optimizations for low-bandwidth environments, making it suitable for deployment in rural regions.

Overall, the system provides a unified, secure, and user-friendly digital healthcare solution that enhances accessibility, reduces dependency on physical consultations, and improves the overall efficiency of healthcare service delivery.

TABLE OF CONTENTS

1. INTRODUCTION	9
1.1 Overview	9
1.2 Problem Statement	10
1.3 Motivation of Theme & Title	10
2. LITERATURE SURVEY	11
3. EXISTING SYSTEM	12
4. PROPOSED SOLUTION	13
4.1 System Design	13
4.1.1 Architecture Diagram	14
4.1.2 Use-Case Diagram	15
4.1.2.1 Use-Case Descriptions	16
4.2 Functional Modules	19
4.2.1 Screenshots	20
4.2.2 Code Implementation	25
5. EXPERIMENTAL SETUP & IMPLEMENTATION	36
5.1 System Specifications	36
5.1.1 Hardware Requirements	36
5.1.2 Software Requirements	36
5.2 Datasets	36
5.3 Methodology / Algorithm	37
6. RESULTS	38
7. CONCLUSION & FUTURE SCOPE	39
8. REFERENCES	41

LIST OF FIGURES

Figure No.	Caption	Page
Figure 4.1	System Architecture Diagram	13
Figure 4.2	Use Case Diagram	14
Figure 4.2.1a	Language Selection Interface	20
Figure 4.2.1b	User Login Page	21
Figure 4.2.1c	User Dashboard	21
Figure 4.2.1d	Doctor List for Booking	22
Figure 4.2.1e	Booking Confirmation	22
Figure 4.2.1f	Symptom Checker	23
Figure 4.2.1g	Medical Records	23
Figure 4.2.1h	Pharmacy Search	24
Figure 4.2.1i	Video Consultation	24
Figure 4.2.1j	Doctor Prescription	25

LIST OF TABLES

Table No.	Caption	Page
Table 5.1	Hardware Requirements	36
Table 5.2	Software Requirements	36

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
JWT	JSON Web Token
API	Application Programming Interface
UI	User Interface
DB	Database
REST	Representational State Transfer
AES	Advanced Encryption Standard
SHA	Secure Hash Algorithm
OTP	One-Time Password
WebRTC	Web Real-Time Communication
i18n	Internationalization

1. INTRODUCTION

1.1 OVERVIEW

Healthcare accessibility continues to be a significant challenge, particularly in rural and semi-urban regions where access to qualified medical professionals, infrastructure, and timely diagnosis is limited. Traditional healthcare systems rely heavily on physical consultations, requiring patients to travel long distances, spend considerable time, and incur additional costs. These challenges often result in delayed treatment and poor health outcomes.

With the rapid advancement of digital technologies, telemedicine has emerged as a promising solution to bridge the gap between patients and healthcare providers. Telemedicine enables remote consultation, diagnosis, and monitoring through digital platforms, thereby reducing the dependency on physical visits. However, existing telemedicine solutions often lack integration, accessibility, and adaptability for rural populations.

The proposed system aims to develop a comprehensive telemedicine platform that integrates multiple healthcare services into a single application. The system allows users to book appointments, consult doctors, analyze symptoms, maintain digital medical records, and search for nearby pharmacies. By combining these functionalities, the platform ensures a seamless healthcare experience for users.

The application is developed using Flutter for the frontend, enabling cross-platform compatibility across mobile and web environments. The backend is built using Node.js and Express, ensuring efficient handling of API requests, while MongoDB is used for flexible and scalable data storage. Additionally, the system incorporates security mechanisms such as password hashing, token-based authentication, and data encryption to ensure patient data privacy. The platform is also designed to function efficiently in low-bandwidth environments, making it suitable for rural deployment.

Overall, the system provides a unified, scalable, and user-friendly digital healthcare solution aimed at improving accessibility, efficiency, and quality of healthcare services.

1.2 PROBLEM STATEMENT

Access to healthcare services in rural areas remains a major concern due to various socio-economic and infrastructural limitations. Patients often face difficulties in reaching hospitals due to long distances, lack of transportation, and limited availability of specialized doctors. This leads to delayed diagnosis and treatment, which can significantly affect patient health outcomes.

Another critical issue is the lack of proper medical record management. Most patients rely on physical records, which can be easily lost or damaged. This creates difficulties in maintaining medical history and continuity of care.

Furthermore, patients often lack the knowledge to identify the appropriate doctor for their symptoms. Without proper guidance, they may consult the wrong specialist, leading to inefficiencies and increased costs.

Existing telemedicine platforms attempt to address these issues but are often limited by their business-oriented models. Many platforms are highly privatized and focus on service monetization rather than accessibility. They may restrict users to specific doctors, pharmacies, or services within their ecosystem, limiting user choice and flexibility.

Additionally, most systems do not provide integrated solutions that combine consultation, diagnosis, and pharmacy services in a single platform. They also lack support for low-bandwidth environments and rural users.

Therefore, there is a need for a comprehensive telemedicine solution that:

- Provides easy access to healthcare services
- Offers intelligent guidance through symptom analysis
- Maintains digital medical records
- Supports both online and offline consultations
- Enables flexible pharmacy search without platform restrictions
- Ensures data security and scalability

1.3 MOTIVATION OF THEME & TITLE

The motivation behind this project is to develop a system that improves healthcare accessibility using digital technology. The focus is on creating a simple, efficient, and scalable solution that can be used by people with limited technical knowledge.

The project aims to integrate consultation, diagnosis, and record management into a single platform, making healthcare services more reachable and effective.

2. LITERATURE SURVEY

The field of telemedicine has evolved significantly over the past decade, driven by advancements in digital technologies and the increasing demand for remote healthcare services. Various telemedicine platforms and research systems have been developed to facilitate online consultations, appointment booking, and digital healthcare management.

Several well-known platforms such as Practo, Apollo 24/7, and other online healthcare applications provide services like doctor consultations, prescription generation, and medicine delivery. These systems have contributed to improving healthcare accessibility, particularly in urban areas where internet connectivity and digital awareness are higher.

In addition to commercial platforms, research studies have explored the integration of intelligent systems into telemedicine applications. Many approaches focus on symptom-based diagnosis using rule-based or machine learning techniques to assist users in identifying potential medical conditions and guiding them toward appropriate healthcare providers.

Despite these advancements, existing telemedicine platforms exhibit certain limitations. Most systems operate within a closed ecosystem, where users are restricted to specific doctors, pharmacies, or services associated with the platform. This limits user flexibility and reduces the ability to make independent healthcare choices.

Furthermore, many platforms are heavily commercialized, requiring users to pay for consultations or purchase medicines through the platform itself. This makes them less accessible to economically weaker sections, especially in rural areas.

Another limitation is the lack of integration between different healthcare services. Many systems treat consultation, record management, and pharmacy services as separate modules, requiring users to switch between multiple applications.

Additionally, several platforms do not incorporate intelligent guidance systems, such as symptom-based doctor recommendation, which can assist users in selecting the appropriate specialist.

The proposed system addresses these gaps by providing an integrated telemedicine platform that combines consultation, symptom analysis, medical records, and pharmacy services while ensuring flexibility, accessibility, and user-centric design.

3. EXISTING SYSTEM

The current healthcare system, especially in rural and semi-urban areas, primarily relies on traditional methods of medical consultation and record management. Patients are required to physically visit hospitals or clinics to seek medical attention, which often leads to delays in diagnosis and treatment. One of the major challenges faced by patients is the lack of access to specialized doctors. Rural areas often have limited healthcare infrastructure, and patients may need to travel long distances to consult specialists. This results in increased time, cost, and inconvenience.

Another significant issue is the absence of proper medical record management. Most patients rely on physical documents to maintain their medical history, which can be easily lost, damaged, or misplaced. This makes it difficult for doctors to access previous records and provide effective treatment.

Patients also face difficulties in identifying the appropriate doctor for their symptoms. Without proper medical knowledge or guidance, they may consult the wrong specialist, leading to inefficient treatment and additional expenses.

In addition, the process of booking appointments is often manual and unorganized. Patients may have to wait in long queues or depend on limited appointment systems, which further delays access to healthcare services.

The availability of medicines is another concern. Patients may not always know where to find required medicines, especially in rural areas where pharmacy access is limited.

Moreover, existing systems are not designed to support low-bandwidth environments, making digital healthcare solutions inaccessible to many rural users.

These limitations highlight the need for a comprehensive telemedicine system that can provide remote consultation, intelligent guidance, digital record management, and improved accessibility, thereby enhancing the overall healthcare experience.

4. PROPOSED SOLUTION

The proposed system is a comprehensive telemedicine platform designed to overcome the limitations of existing healthcare systems. It integrates multiple healthcare services into a single application, providing users with a seamless and efficient experience.

The system enables users to:

- Book online and offline appointments
- Consult doctors remotely
- Analyze symptoms using a rule-based system
- Maintain digital medical records
- Search for nearby pharmacies

The platform is designed to be user-friendly, lightweight, and scalable, ensuring accessibility for users with varying levels of technical knowledge.

Unlike existing systems, the proposed solution focuses on flexibility and user-centric design. Users are not restricted to a specific ecosystem and can freely choose doctors and pharmacies based on their requirements.

The system also incorporates security mechanisms such as encryption and authentication to ensure data privacy and protection.

4.1 SYSTEM DESIGN

The system design follows a layered client-server architecture to ensure modularity, scalability, and efficient data handling. The architecture is divided into multiple layers, including the presentation layer, application layer, and data layer.

The presentation layer is developed using Flutter and is responsible for handling user interactions, displaying information, and communicating with the backend through REST APIs. This layer provides an intuitive and user-friendly interface for both patients and doctors.

The application layer is implemented using Node.js and Express. It handles all business logic, including authentication, appointment management, symptom analysis, medical record processing, and pharmacy search. The backend acts as an intermediary between the frontend and the database, ensuring proper validation and processing of requests.

The data layer uses MongoDB as the database, which stores all relevant data such as user details, doctor information, appointments, medical records, and pharmacy data. MongoDB's flexible schema allows efficient handling of structured and semi-structured data.

4.1.1 ARCHITECTURE DIAGRAM

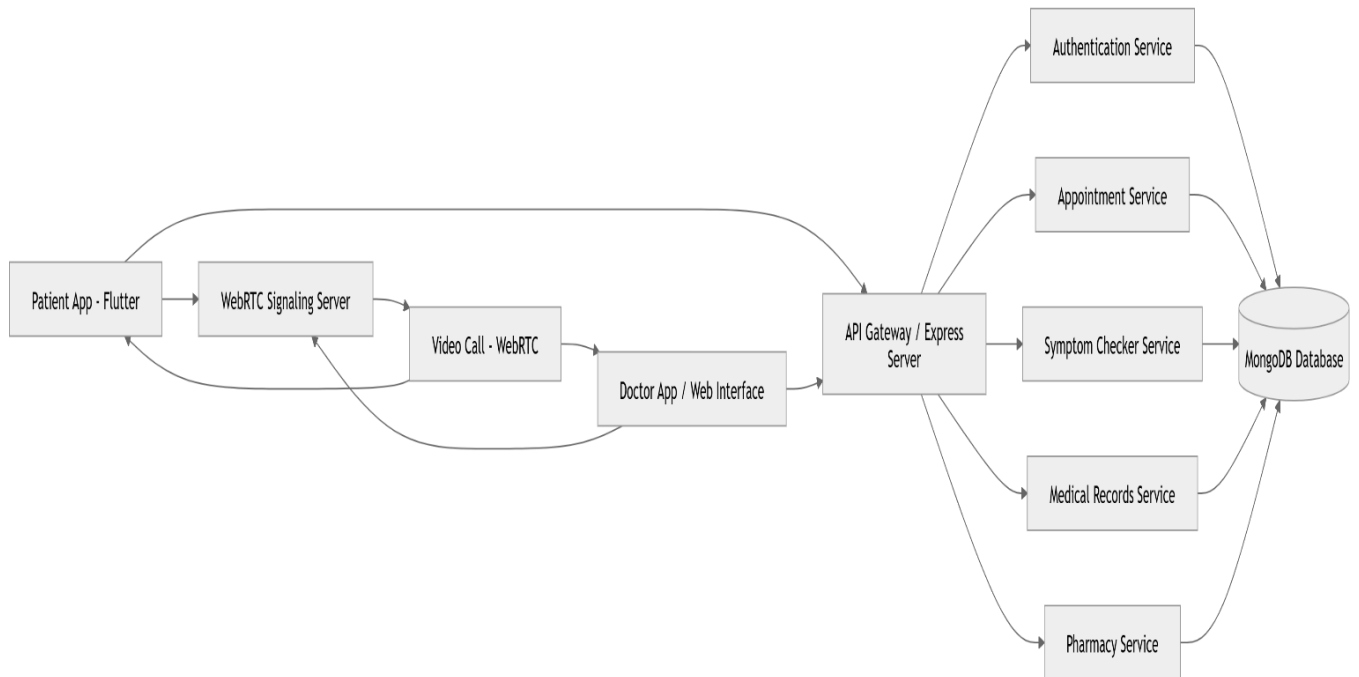


Figure 4.1 System Architecture

The architecture diagram illustrates the interaction between different components of the telemedicine system. The frontend, developed using Flutter, communicates with the backend server through REST APIs. The backend processes requests using various services such as authentication, appointment management, symptom analysis, and medical records handling.

MongoDB is used as the central database to store all application data. The system also includes a WebRTC-based communication layer for enabling real-time consultation between patients and doctors. This architecture ensures efficient data flow, modular design, and scalability, allowing the system to handle multiple users and services effectively.

4.1.2 USE CASE DIAGRAM

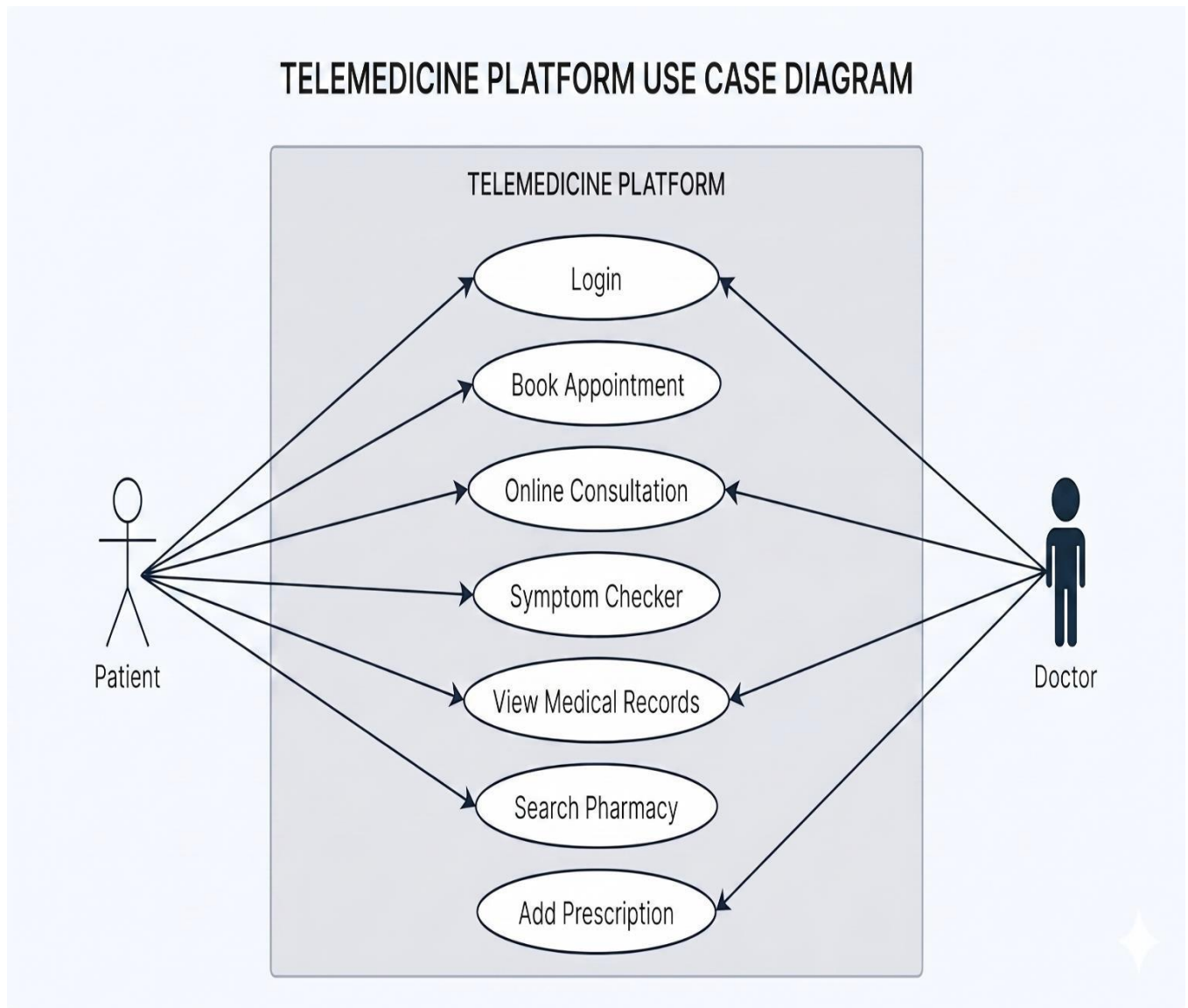


Figure 4.2 Use Case Diagram

The use case diagram illustrates the interaction between the primary actors, namely the patient and the doctor, with the telemedicine system. The patient can perform actions such as login, booking appointments, symptom checking, viewing medical records, and searching for pharmacies. The doctor is responsible for consultation and adding prescriptions. The diagram represents the overall functionality and user interactions within the system.

4.1.2.1 USE CASE DESCRIPTIONS

UC-001: User Login

Use Case ID	UC-001
Use Case Name	User Login
Actor(s)	Patient, Doctor
Precondition	User must be registered in the system.
Main Flow	1. User enters phone number and password. 2. System validates user credentials. 3. Password is verified using bcrypt hashing. 4. System generates JWT token. 5. User is redirected to dashboard.
Alternative Flow	2a: User enters incorrect credentials — system displays error message.
Exception Flow	3a: Server error — system displays login failure message.
Postcondition	User is successfully authenticated and granted access to the system.
Includes	Dashboard Access

UC-002: Book Appointment

Use Case ID	UC-002
Use Case Name	Appointment Booking
Actor(s)	Patient
Precondition	User must be logged in.
Main Flow	1. User selects doctor from list. 2. User selects date and available time slot. 3. System checks slot availability. 4. System confirms booking. 5. Appointment is stored in database.
Alternative Flow	2a: Selected slot is unavailable — system prompts user to choose another slot.
Exception Flow	3a: Database error — booking fails and error message is shown.
Postcondition	Appointment is successfully booked and slot is reserved.
Includes	Doctor Selection

UC-003: Online Consultation

Use Case ID	UC-003
Use Case Name	Online Doctor Consultation
Actor(s)	Patient, Doctor
Precondition	Appointment must exist.
Main Flow	1. Patient selects online consultation option. 2. System connects patient and doctor. 3. Consultation session begins. 4. Doctor interacts with patient. 5. Consultation ends.
Alternative Flow	2a: Connection issue — system retries connection.
Exception Flow	3a: Network failure — consultation terminated.
Postcondition	Consultation is completed successfully.
Includes	Add Prescription

UC-004: Add Prescription

Use Case ID	UC-004
Use Case Name	Prescription Entry
Actor(s)	Doctor
Precondition	Consultation must be completed.
Main Flow	1. Doctor enters medicine details. 2. Doctor specifies dosage and timing. 3. System validates input. 4. Prescription is stored in database.
Alternative Flow	2a: Doctor edits prescription before submission.
Exception Flow	3a: Invalid input — system displays validation error.
Postcondition	Prescription is saved and linked to patient records.
Includes	Medical Records Update

UC-005: Symptom Checker

Use Case ID	UC-005
Use Case Name	Symptom Analysis
Actor(s)	Patient
Precondition	User must be logged in
Main Flow	1. User selects symptoms from list. 2. System processes symptoms using rule-based logic. 3. System identifies possible condition. 4. System suggests doctor specialization.
Alternative Flow	1a: User selects multiple symptoms — system prioritizes results.
Exception Flow	2a: No matching rule — system suggests general physician.
Postcondition	User receives condition and doctor recommendation.
Includes	Doctor Recommendation

UC-006: View Medical Records

Use Case ID	UC-006
Use Case Name	Medical Records
Actor(s)	Patient, Doctor
Precondition	Records must exist in system.
Main Flow	1. User accesses medical records section. 2. System fetches data from database. 3. Records are displayed to user.
Alternative Flow	2a: No records found — system displays empty state.
Exception Flow	2b: Database error — system shows error message.
Postcondition	User successfully views medical records.
Includes	Prescription Details

UC-007: Search Pharmacy

Use Case ID	UC-007
Use Case Name	Pharmacy Search
Actor(s)	Patient
Precondition	User must enter medicine name.
Main Flow	1. User enters medicine name. 2. System searches database. 3. System retrieves matching pharmacies. 4. Results are displayed to user.
Alternative Flow	2a: No matching pharmacy — system shows no results
Exception Flow	2b: Server error — system displays error message.
Postcondition	User receives list of pharmacies with medicine availability.

4.2 FUNCTIONAL MODULES

The telemedicine system is divided into multiple functional modules to ensure efficient operation, modular design, and ease of maintenance. Each module handles a specific functionality of the system, allowing better organization and scalability.

Module 1:Authentication Module

The authentication module is responsible for managing user registration and login processes. Users can create an account by providing details such as name, phone number, and password. The password is securely hashed using bcrypt before being stored in the database.

During login, the entered credentials are validated by comparing the password with the stored hashed value. Upon successful authentication, a JWT token is generated and used for maintaining user sessions. This module ensures secure access and prevents unauthorized usage of the system.

Module 2:Doctor Management Module

The doctor module maintains information related to doctors, including their name, specialization, and availability. This data is stored in the database and is dynamically fetched when users want to book appointments.

Users can view a list of available doctors along with their specialization and choose the appropriate doctor based on their requirement. The availability of doctors is managed in terms of dates and time slots, ensuring accurate scheduling.

Module 3:Appointment Module

The appointment module handles both online and offline consultation bookings. For offline appointments, users can select a doctor, choose a date, and pick an available time slot. Once the booking is confirmed, the selected slot is removed from the available list to prevent double booking. For online consultations, users can initiate a consultation session with a doctor. After the consultation, the details are recorded and stored in the system. This module ensures smooth scheduling and management of appointments.

Module 4:Symptom Checker Module

The symptom checker module provides basic medical guidance to users based on their symptoms. Users can select symptoms through the application interface, and the system processes these inputs using rule-based logic.

Based on the selected symptoms, the system identifies possible conditions and recommends the appropriate type of doctor. This helps users make informed decisions and reduces the chances of consulting the wrong specialist.

Module 5:Medical Records Module

The medical records module is responsible for storing and managing patient-related information. It includes consultation history, prescriptions, and appointment details.

All records are stored securely in the database, and users can access them at any time. This ensures proper maintenance of medical history and helps doctors provide better treatment based on past records.

Module 6:Prescription Module

The prescription module allows doctors to add prescriptions after completing a consultation. The prescription includes details such as medicine name, dosage, and timing instructions.

These prescriptions are stored in the medical records and can be viewed by the patient later. This module ensures that all treatment details are digitally maintained for future reference.

Module 7:Pharmacy Module

The pharmacy module enables users to search for medicines and find nearby pharmacies where the required medicines are available. Users can enter the medicine name, and the system returns relevant results.

This feature improves accessibility by helping users quickly locate medicines after consultation. Future improvements include integration with location-based services for real-time tracking.

4.2.1 SCREENSHOTS

This screen allows users to select their preferred language (English, Hindi, Telugu), enabling multilingual accessibility for rural users.

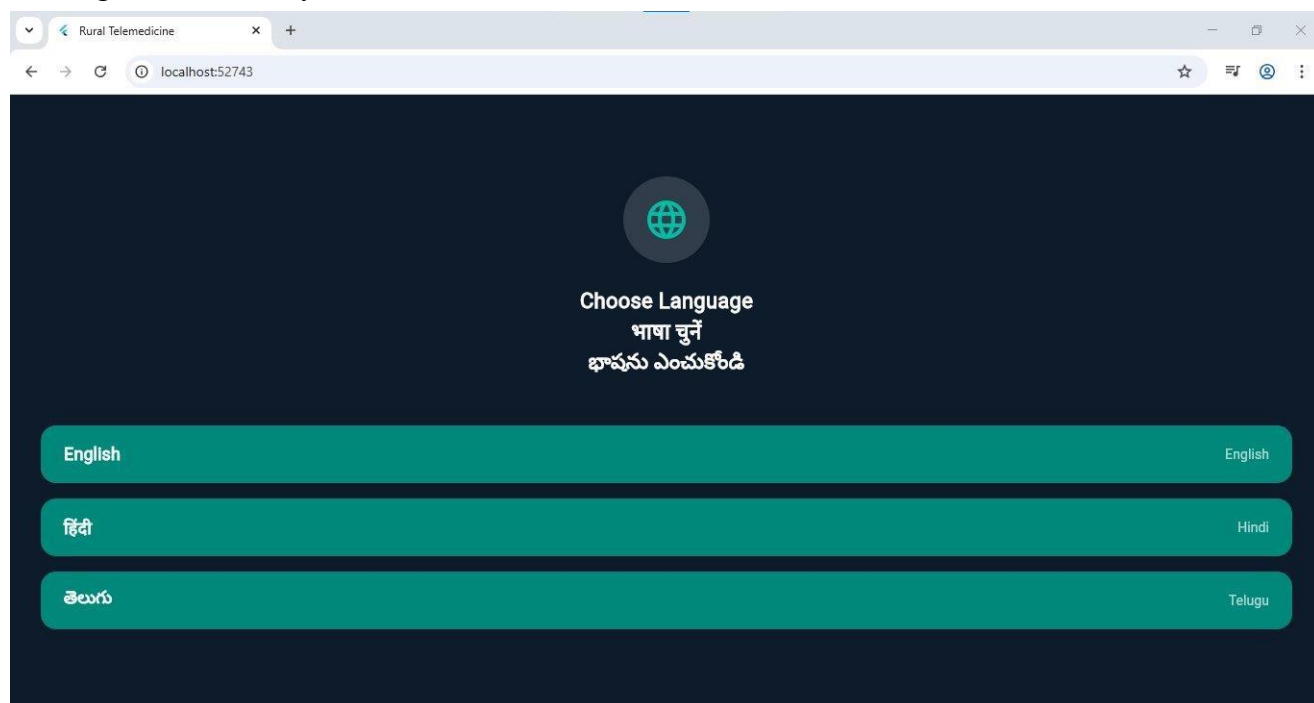


Figure 4.2.1a – Language Selection Interface

This interface enables users to log in securely using their credentials. It includes input fields for email and password.

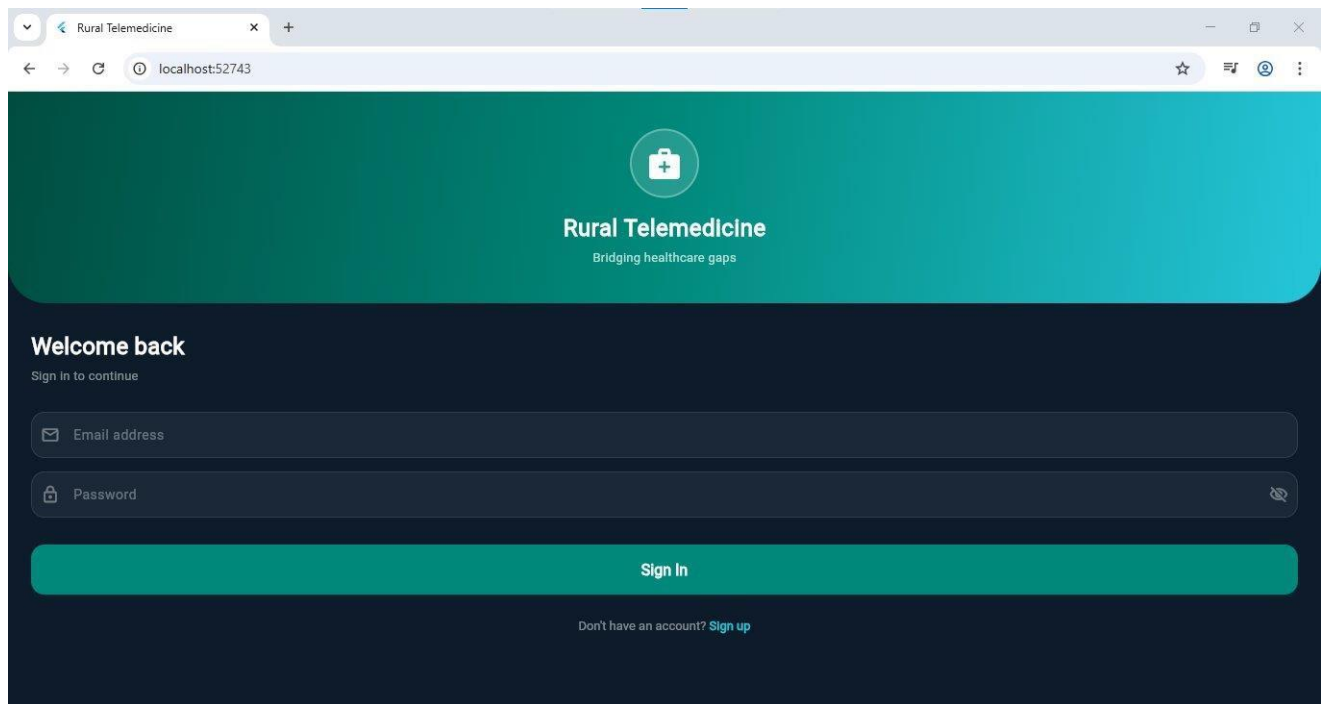


Figure 4.2.1b – User Login Page

This screen displays the dashboard with quick access to features like appointments, symptom checker, and medical records.

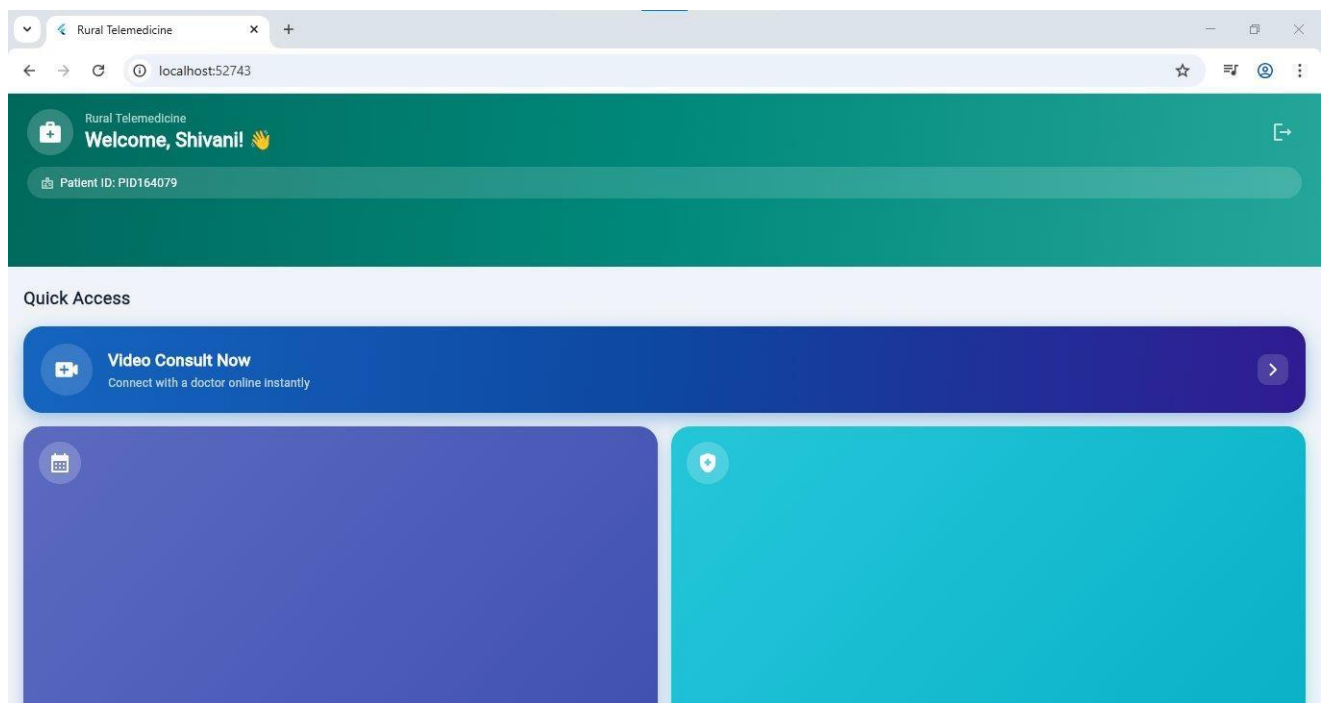


Figure 4.2.1c – User Dashboard

This interface shows available doctors along with specialization and availability.

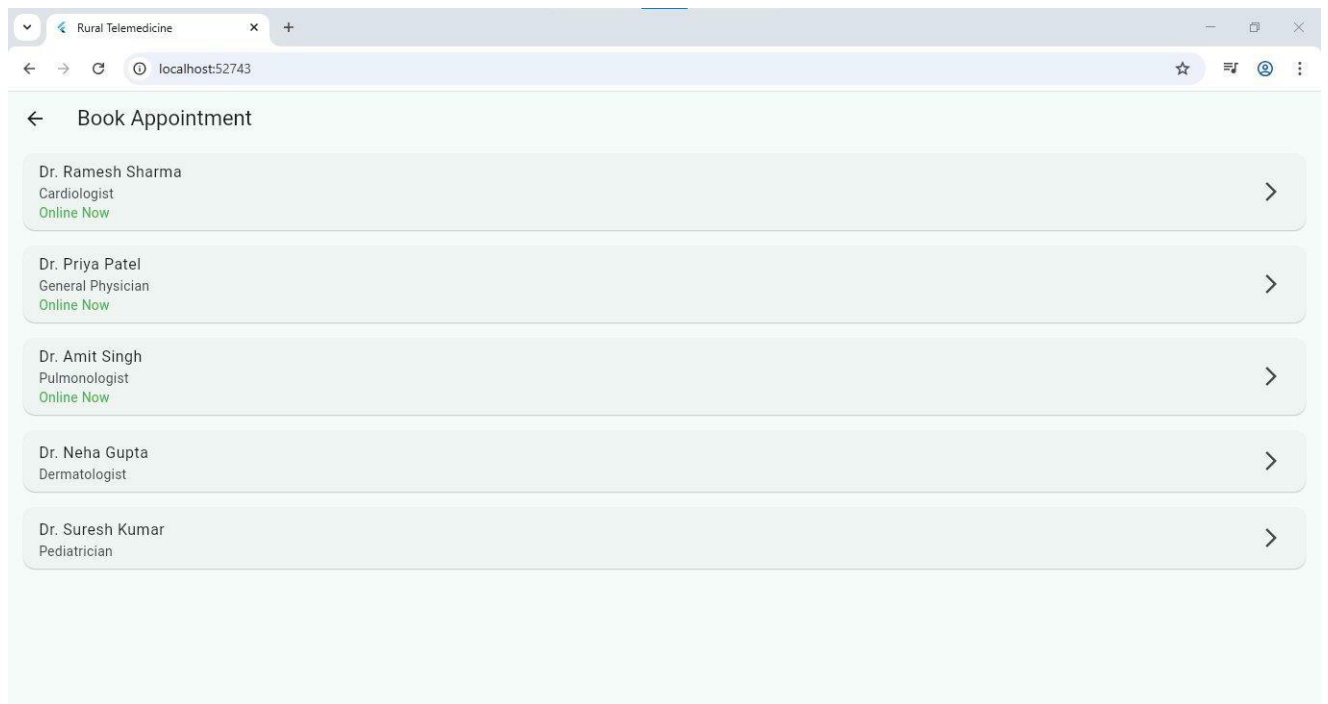


Figure 4.2.1d– Doctor List for Booking

This screen displays appointment booking details including doctor, date, and time.

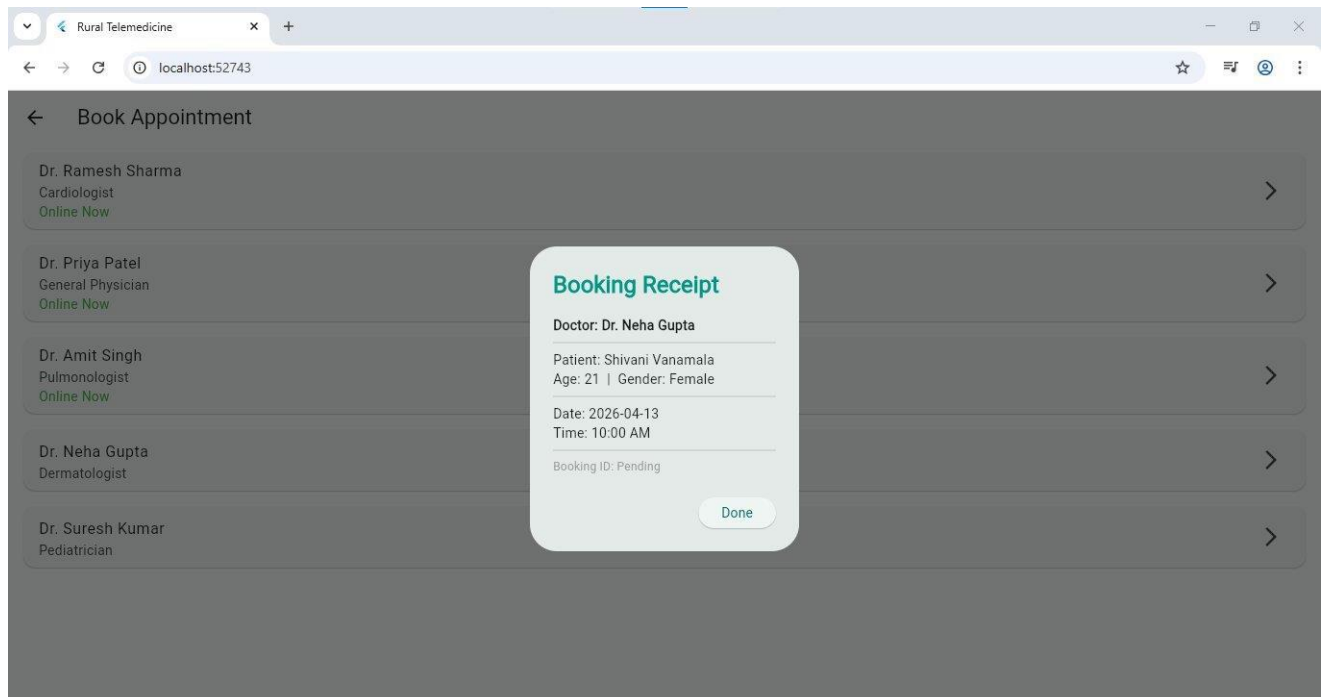


Figure 4.2.1e – Booking Confirmation

This screen allows users to select symptoms and get doctor recommendations

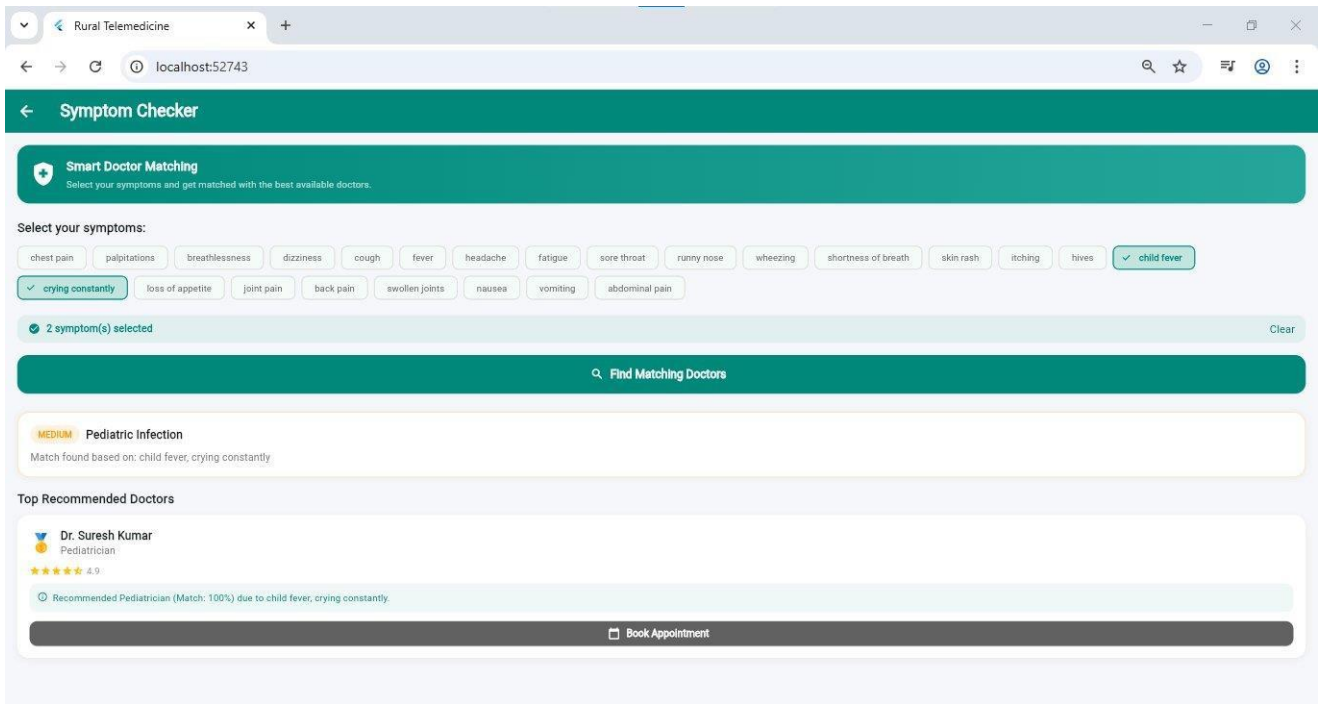


Figure 4.2.1f – Symptom Checker

This interface shows consultation history and prescriptions.

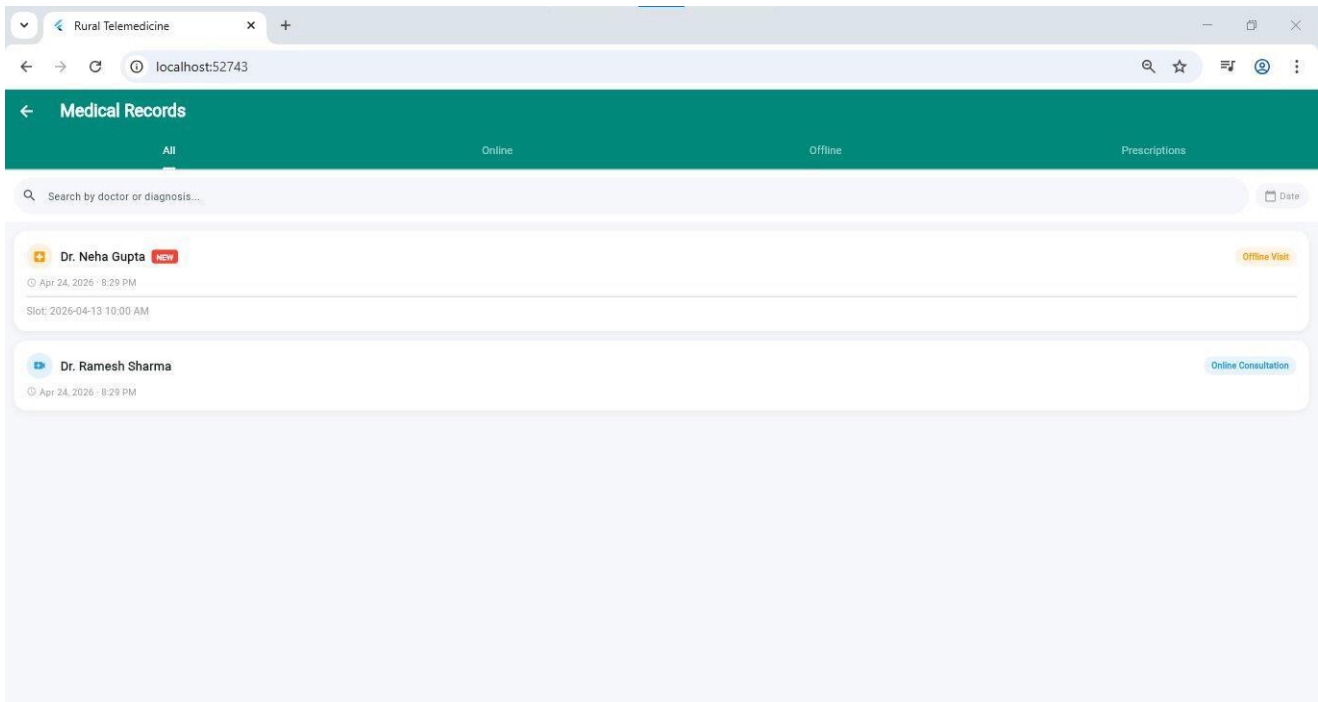


Figure 4.2.1g – Medical records

This screen displays nearby pharmacies and available medicines.

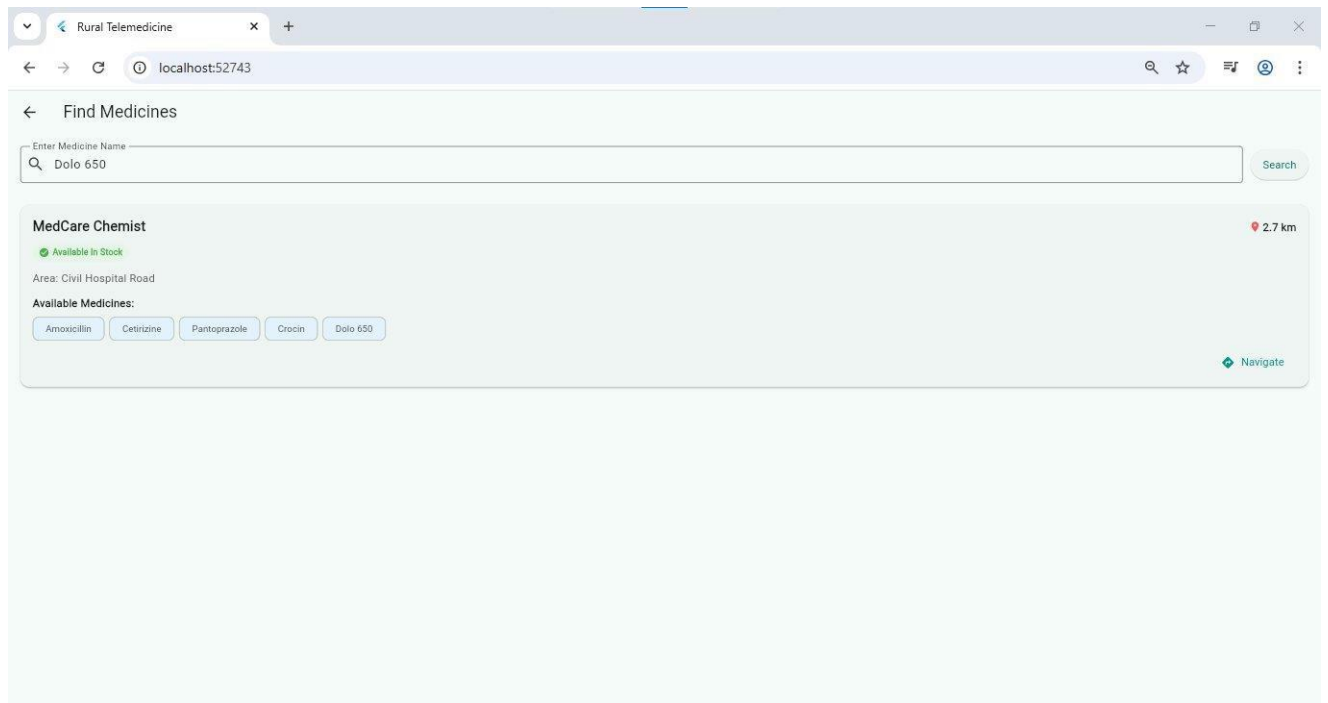


Figure 4.2.1h – Pharmacy Search

This interface enables real-time doctor-patient communication using WebRTC.

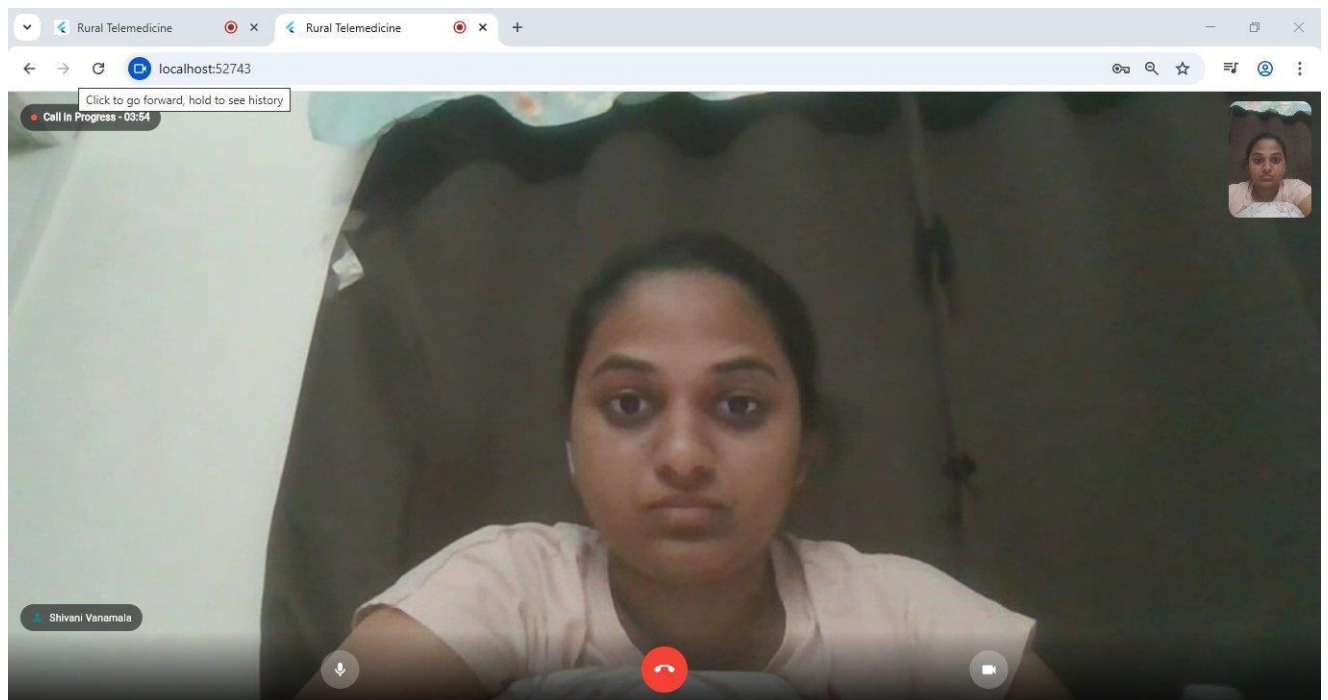


Figure 4.2.1i – Video Consultation

This Screen enables doctors to give digital prescription after online consultation

The screenshot shows a web browser window with two tabs labeled 'Rural Telemedicine'. The address bar shows 'localhost:52743'. The page title is 'Write Prescription'. At the top, there is a patient information bar for 'Shivani Vanamala'. Below this, the 'Diagnosis' field contains 'Viral Fever'. The 'Medicines' section has a table with one entry: 'Dolo' with a dosage of '650mg' and a duration of '3 Days'. There is an 'Add Medicine' button. The 'Doctor's Notes' field contains 'Avoid Junk Food'. At the bottom, there is a green button labeled 'Save Prescription'.

Figure 4.2.1j – Doctor Prescription

4.2.2 CODE IMPLEMENTATION

The code below describes the core algorithmic logic of key functional modules.

Code 1 – User Authentication

```
backend > middleware > JS authMiddleware.js > ...
1  const jwt = require('jsonwebtoken');
2
3  const protect = (req, res, next) => {
4    let token;
5
6    if (req.headers.authorization && req.headers.authorization.startsWith('Bearer ')) {
7      try {
8        token = req.headers.authorization.split(' ')[1];
9        // Decoded now contains: { id, role }
10       const decoded = jwt.verify(token, process.env.JWT_SECRET || 'fallback_secret');
11       req.user = decoded;
12       next();
13     } catch (error) {
14       console.error(error);
15       res.status(401).json({ message: 'Not authorized, token failed' });
16     }
17   }
18
19   if (!token) {
20     res.status(401).json({ message: 'Not authorized, no token' });
21   }
22 };
23
24 // Doctor-only middleware – chains protect then checks role
25 const protectDoctor = (req, res, next) => {
26   protect(req, res, () => {
27     if (req.user && req.user.role === 'doctor') {
28       next();
29     } else {
30       res.status(403).json({ message: 'Access denied: doctor role required' });
31     }
32   });
33 };
34
35 module.exports = { protect, protectDoctor };
```

Code 2 – Login API Integration

```
29
30 // — Auth —————
31
32 static Future<Map<String, dynamic>> login(
33     String email, String password) async {
34     final response = await http.post(
35         Uri.parse('$baseUrl/auth/login'),
36         headers: {'Content-Type': 'application/json'},
37         body: jsonEncode({'email': email, 'password': password}),
38     );
39     return jsonDecode(response.body);
40 }
```

Code 3 – Fetch Doctors

```
static Future<List<dynamic>> getDoctors() async {
    final headers = await getDefaultHeaders();
    final response =
        await http.get(Uri.parse('$baseUrl/doctors'), headers: headers);
    if (response.statusCode == 200) return jsonDecode(response.body);
    throw Exception('Failed to load doctors');
}

static Future<List<dynamic>> getDoctorsBySpecialization(
    String specialization) async {
    final headers = await getDefaultHeaders();
    final response = await http.get(
        Uri.parse(
            '$baseUrl/doctors/by-specialization?specialization=$specialization'),
        headers: headers,
    );
    if (response.statusCode == 200) return jsonDecode(response.body);
    throw Exception('Failed to load doctors by specialization');
}
```

Code 4 – Book Appointment (Offline)

```
// — Appointments —

static Future<Map<String, dynamic>> bookOffline(String doctorId, String date,
    String slot, String pName, int pAge, String pGender) async {
  final headers = await getDefaultHeaders();
  final response = await http.post(
    Uri.parse('$baseUrl/appointments/book'),
    headers: headers,
    body: jsonEncode({
      'doctorId': doctorId,
      'date': date,
      'slot': slot,
      'patientName': pName,
      'patientAge': pAge,
      'patientGender': pGender
    })),
  );
  return jsonDecode(response.body);
}

static Future<Map<String, dynamic>> bookOnline(
    String doctorId, String pName, int pAge, String pGender) async {
  final headers = await getDefaultHeaders();
  final response = await http.post(
    Uri.parse('$baseUrl/appointments/online-consult'),
    headers: headers,
    body: jsonEncode({
      'doctorId': doctorId,
      'patientName': pName,
      'patientAge': pAge,
      'patientGender': pGender
    })),
  );
  return jsonDecode(response.body);
}
```

Code 5 – Online Consultation (Video Call)

```
_localStream = await navigator.mediaDevices.getUserMedia({
  'audio': true,
  'video': {'facingMode': 'user', 'width': 1280, 'height': 720},
});
if (mounted) {
  setState(() {
    _localRenderer.srcObject = _localStream;
    _status = 'Waiting for patient...';
  });
}

_peerConnection = await createPeerConnection(_rtcConfig);

_localStream!.getTracks().forEach((t) {
  _peerConnection!.addTrack(t, _localStream!);
});
```

```
_peerConnection!.onTrack = (RTCTrackEvent event) {
  if (event.streams.isNotEmpty && mounted) {
    setState(() {
      _remoteRenderer.srcObject = event.streams[0];
      _callConnected = true;
      _status = 'Connected';
      _startTimer();
    });
  }
};
```

Code 6 - Symptom Checker

```
static Future<Map<String, dynamic>> analyzeSymptoms(
    List<String> symptoms) async {
    final headers = await getDefaultHeaders();
    final response = await http.post(
        Uri.parse('$baseUrl/symptoms/analyze'),
        headers: headers,
        body: jsonEncode({'symptoms': symptoms}),
    );
    return jsonDecode(response.body);
}
```

Code 7 – Add Prescription

```
static Future<Map<String, dynamic>> submitPrescription({
    required String patientId,
    String? appointmentId,
    required String diagnosis,
    required List<Map<String, String>> medicines,
    String notes = '',
}) async {
    final headers = await getDefaultHeaders();
    final response = await http.post(
        Uri.parse('$baseUrl/prescriptions'),
        headers: headers,
        body: jsonEncode({
            'patientId': patientId,
            if (appointmentId != null) 'appointmentId': appointmentId,
            'diagnosis': diagnosis,
            'medicines': medicines,
            'notes': notes,
        })),
    );
    return jsonDecode(response.body);
}
```


Code 8 – View Medical Records

```
// — Medical Records —————

static Future<List<dynamic>> getRecords(String patientId,
    {String? search, String? date, String? category}) async {
  final headers = await getDefaultHeaders();
  final params = <String, String>{};
  if (search != null && search.isNotEmpty) params['search'] = search;
  if (date != null && date.isNotEmpty) params['date'] = date;
  if (category != null && category.isNotEmpty) params['category'] = category;
  final uri = Uri.parse('$baseUrl/records/$patientId')
    .replace(queryParameters: params);
  final response = await http.get(uri, headers: headers);
  if (response.statusCode == 200) return jsonDecode(response.body);
  throw Exception('Failed to load records');
}
```

Code 9 – Search Pharmacy

```
// — Pharmacies —————

static Future<List<dynamic>> searchPharmacies(String medicine) async {
  final headers = await getDefaultHeaders();
  final response = await http.get(
    Uri.parse('$baseUrl/pharmacies/search?medicine=$medicine'),
    headers: headers,
  );
  if (response.statusCode == 200) return jsonDecode(response.body);
  throw Exception('Failed to search pharmacies');
}
```

Code 10-Doctor Availability

```
static Future<Map<String, dynamic>> toggleDoctorAvailability(
    String doctorId) async {
    final headers = await getDefaultHeaders();
    final response = await http.patch(
        Uri.parse('$baseUrl/doctors/$doctorId/availability'),
        headers: headers,
    );
    if (response.statusCode == 200) return jsonDecode(response.body);
    throw Exception('Failed to toggle availability');
}
```

Code 11:Authentication Token

```
static Future<String?> getToken() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString('token');
}
```

```
static Future<Map<String, String>> getDefaultHeaders() async {
    final token = await getToken();
    final headers = {'Content-Type': 'application/json'};
    if (token != null) headers['Authorization'] = 'Bearer $token';
    return headers;
}
```

Code 12-Doctor Filtering

```
// @route GET /api/doctors/by-specialization
const getDoctorsBySpecialization = async (req, res) => {
  try {
    const { specialization } = req.query;
    let query = {};
    if (specialization) {
      query.specialization = specialization;
    }
    const doctors = await Doctor.find(query);
    res.json(doctors);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Code 13-Call Session

```
void _connectSocket() {
  _socket = IO.io(
    ApiService.socketUrl,
    IO.OptionBuilder().setTransports(['websocket']).disableAutoConnect().build(),
  );
  _socket.connect();

  _socket.onConnect((_) {
    setState(() => _status = 'Joining room...');
    _socket.emit('join-room', widget.appointmentId);
  });
}
```

Code 14-ICE Candidate Exchange

```
peerConnection!.onIceCandidate = (RTCIceCandidate? candidate) {
  if (candidate != null) {
    socket.emit('ice-candidate', {
      'roomId': widget.appointmentId,
      'candidate': candidate.toMap(),
    });
  }
};

_connectSocket();
}
```

Code 15-Secure API Communication

```
static Future<Map<String, String>> getDefaultHeaders() async {  
  final token = await getToken();  
  final headers = {'Content-Type': 'application/json'};  
  if (token != null) headers['Authorization'] = 'Bearer $token';  
  return headers;  
}
```

Code 16-Real-Time Signaling

```
// Patient sends offer → doctor creates answer  
_socket.on('offer', (data) async {  
  if (mounted) setState(() => _status = 'Answering call...');  
  final sdp = RTCSessionDescription(data['sdp']['sdp'], data['sdp']['type']);  
  await _peerConnection?.setRemoteDescription(sdp);  
  final answer = await _peerConnection!.createAnswer();  
  await _peerConnection!.setLocalDescription(answer);  
  _socket.emit('answer', {  
    'roomId': widget.appointmentId,  
    'sdp': answer.toMap(),  
  });  
});
```


Code 17-Media stream Handling

```
_localStream = await navigator.mediaDevices.getUserMedia({
  'audio': true,
  'video': {'facingMode': 'user', 'width': 1280, 'height': 720},
});
if (mounted) {
  setState(() {
    _localRenderer.srcObject = _localStream;
    _status = 'Waiting for patient...';
  });
}

_peerConnection = await createPeerConnection(_rtcConfig);

_localStream!.getTracks().forEach((t) {
  _peerConnection!.addTrack(t, _localStream!);
});
```

```
static Future<Map<String, dynamic>> signup(String name, String email,
  String password, String phone, String age, String gender) async {
  final response = await http.post(
    Uri.parse('$baseUrl/auth/signup'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({
      'name': name,
      'email': email,
      'password': password,
      'phone': phone,
      'age': int.parse(age),
      'gender': gender
    })),
  );
  return jsonDecode(response.body);
}
```

Code 18-Session Timer

```
void _startTimer() {
  _timer?.cancel();
  _timer = Timer.periodic(const Duration(seconds: 1), (timer) {
    if (mounted) setState(() => _secondsElapsed++);
  });
}

String _formatTime(int totalSeconds) {
  int minutes = totalSeconds ~/ 60;
  int seconds = totalSeconds % 60;
  return '${minutes.toString().padLeft(2, '0')}:${seconds.toString().padLeft(2, '0')}';
}
```

Code 19-Post Consultation Navigation

```
void _onPatientLeft() => _cleanupAndNavigate();

void _cleanupAndNavigate() {
  _timer?.cancel();
  _peerConnection?.close();
  _localStream?.dispose();
  _socket.dispose();
  if (mounted) {
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(
        builder: (_) => PrescriptionFormScreen(
          patientId: widget.patientId,
          patientName: widget.patientName,
          appointmentId: widget.appointmentId,
        ),
      ),
    );
  }
}
```

Code 20-Error Handling

```
    if (response.statusCode == 200) return jsonDecode(response.body);  
    throw Exception('Failed to load doctors by specialization');  
}  
  
static Future<Map<String, dynamic>> toggleDoctorAvailability(  
    String doctorId) async {  
    final headers = await getDefaultHeaders();  
    final response = await http.patch(  
        Uri.parse('$baseUrl/doctors/$doctorId/availability'),  
        headers: headers,  
    );  
    if (response.statusCode == 200) return jsonDecode(response.body);  
    throw Exception('Failed to toggle availability');  
}
```

5. EXPERIMENTAL SETUP & IMPLEMENTATION

5.1 SYSTEM SPECIFICATIONS

5.1.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel i3 or equivalent
RAM	4 GB
Storage	10 GB free space
Device	Laptop / Desktop
Mobile Device (Optional)	Android device for testing
Network	Internet connection

Table 5.1

5.1.2 Software Requirements

Software	Version / Details
Flutter SDK	Latest Stable Version
Node.js	v16 or above
MongoDB	v5.0 or above
Visual Studio Code	Latest Version
Web Browser	Google Chrome (latest)
Thunder Client	API Testing Tool

Table 5.2

5.2 DATASETS

The proposed system does not rely on external datasets but utilizes internally created and managed data stored in the MongoDB database.

The following datasets are used within the system:

- **Doctor Dataset:** Contains doctor details such as name, specialization, and available time slots.
- **Symptom Rules Dataset:** Includes predefined rules mapping symptoms to possible conditions and recommended specializations.
- **Pharmacy Dataset:** Stores information about medicines and their availability in nearby pharmacies.

5.3 METHODOLOGY / ALGORITHM

The system follows a structured pipeline consisting of multiple phases:

Phase 1: User Authentication

- User enters login credentials
- Password is verified using bcrypt
- JWT token is generated for authentication
- User is redirected to dashboard

Phase 2: Doctor Retrieval

- System fetches doctor data from database
- Displays list of doctors with specialization and availability

Phase 3: Symptom Analysis

- User selects symptoms
- System matches symptoms with predefined rules
- Determines possible condition and specialization
- Recommends suitable doctor

Phase 4: Appointment Booking

- User selects doctor, date, and slot
- System checks slot availability
- If available → booking is confirmed
- Slot is removed to prevent duplication

Phase 5: Consultation

- User initiates online consultation (simulated WebRTC)
- Doctor interacts with patient
- Consultation details are processed

Phase 6: Prescription Generation

- Doctor enters prescription details
- Prescription is stored in medical records

Phase 7: Medical Records Management

- System stores consultation history and prescriptions
- Users can view records anytime

Phase 8: Pharmacy Search

- User enters medicine name
- System searches database
- Displays nearby pharmacies

6. RESULTS

The telemedicine platform was evaluated across multiple functional dimensions, including authentication reliability, appointment management efficiency, symptom analysis accuracy, and medical record handling. The system was tested under various user scenarios to ensure functionality, usability, and performance.

Authentication & Security Performance

The authentication module was tested with multiple user login and registration scenarios. Passwords were securely hashed using bcrypt, and login validation was performed using hashed comparisons. JWT-based authentication ensured secure session management across all API requests.

The system successfully handled both valid and invalid login attempts, preventing unauthorized access. Token generation and validation were consistent across sessions, ensuring reliable and secure user authentication.

Appointment Management Efficiency

The appointment module was tested with multiple booking scenarios involving different doctors, dates, and time slots. The system correctly displayed available slots and allowed users to book appointments without conflicts.

Once a slot was booked, it was dynamically removed from the available list, ensuring no double booking occurred. Both online and offline consultation workflows were executed successfully, demonstrating the system's ability to manage scheduling efficiently.

Symptom Checker Accuracy

The symptom checker module was evaluated using various combinations of user-input symptoms. The rule-based system successfully mapped symptoms to relevant medical conditions and recommended appropriate doctor specializations.

For example, symptoms such as chest pain and breathlessness correctly resulted in cardiology-related recommendations, while general symptoms like fever and fatigue mapped to general physician suggestions. This demonstrated the effectiveness of the rule-based approach in guiding users.

Medical Records Management

The medical records module was tested for storing and retrieving patient data, including prescriptions and consultation history. The system successfully stored all records in MongoDB and allowed users to access them without data loss.

Sensitive data was secured using hashing techniques, ensuring data privacy. The module demonstrated consistency and reliability in maintaining patient history.

7. CONCLUSION & FUTURE SCOPE

7.1 CONCLUSION

The telemedicine platform developed in this project demonstrates how digital technologies can enhance healthcare accessibility, efficiency, and reliability, especially in rural and underserved areas. The system integrates key functionalities such as appointment booking, symptom analysis, medical record management, and pharmacy search into a single unified application, providing a seamless and user-friendly experience.

A major achievement of this project is reducing dependency on traditional healthcare systems that require physical presence. By supporting both online and offline consultations, the system offers flexibility based on user connectivity and accessibility. The rule-based symptom checker further assists users in identifying the appropriate doctor, minimizing incorrect consultations and saving time and cost. The implementation of secure authentication using bcrypt and JWT ensures data privacy and protection. Additionally, hashing techniques for sensitive medical data enhance overall system security, which is critical in healthcare applications.

The system architecture, built using Flutter, Node.js, and MongoDB, ensures scalability, modularity, and ease of maintenance. The inclusion of WebRTC (simulated) demonstrates the system's ability to support real-time communication.

Testing across various modules confirmed that the system performs efficiently and delivers accurate results. The user interface is simple and intuitive, making it accessible even for users with limited technical knowledge.

Overall, the project successfully achieves its objective of providing a secure, scalable, and efficient telemedicine solution, highlighting the potential of digital healthcare systems in improving accessibility and service quality.

7.2 FUTURE SCOPE

The proposed system provides a strong foundation for future enhancements and real-world deployment. Several improvements can be implemented to make the system more advanced and effective.

Integration with Google Maps API can enable real-time location-based pharmacy and hospital search, improving usability. Full implementation of WebRTC can provide real-time video consultation, making the platform more interactive and practical.

The system can be enhanced using artificial intelligence and machine learning for more accurate symptom analysis and personalized doctor recommendations. A dedicated doctor-side application can also be developed to improve consultation management and workflow efficiency.

Expanding multilingual support will make the system more accessible to users from diverse linguistic backgrounds, particularly in rural areas. Additional features such as real-time notifications, appointment reminders, and emergency alerts can further improve user engagement.

From a scalability perspective, the system can be extended using cloud infrastructure, load balancing, and distributed databases to support a larger user base. Integration with wearable devices can enable continuous health monitoring and proactive healthcare management.

In conclusion, the system has significant potential for growth and can evolve into a fully functional, intelligent, and scalable telemedicine platform capable of addressing real-world healthcare challenges.

8. REFERENCES

- [1] Flutter Team, "Flutter Documentation – Build apps for any screen," Google, 2025.
[Online]. Available: <https://docs.flutter.dev>
- [2] Node.js Foundation, "Node.js Documentation," OpenJS Foundation, 2025.
[Online]. Available: <https://nodejs.org/en/docs>
- [3] Express.js, "Express – Fast, unopinionated, minimalist web framework for Node.js," 2025. [Online]. Available: <https://expressjs.com>
- [4] MongoDB Inc., "MongoDB Atlas Documentation," 2025.
[Online]. Available: <https://www.mongodb.com/docs>
- [5] Auth0, "JSON Web Token (JWT) Introduction," 2025.
[Online]. Available: <https://jwt.io>
- [6] npm Inc., "bcrypt.js Library Documentation," 2025.
[Online]. Available: <https://www.npmjs.com/package/bcryptjs>
- [7] WebRTC Project, "WebRTC – Real-Time Communication in Browsers," 2025.
[Online]. Available: <https://webrtc.org>
- [8] Mozilla Developer Network, "HTTP and REST API Documentation," 2025.
[Online]. Available: <https://developer.mozilla.org>
- [9] Google Developers, "Google Maps Platform Documentation," 2025.
[Online]. Available: <https://developers.google.com/maps>
- [10] World Health Organization (WHO), "Telemedicine: Opportunities and Developments in Member States," 2024. [Online]. Available: <https://www.who.int/health-topics/telemedicine>